PRESENTATION SCRIPT

AI-Powered Resume Screening and Ranking System

Major Project Phase II | Dept. of ISE, VKIT

Tip: speak slowly, about 30 seconds per slide (12 to 14 minutes in total). Say the bold line of each slide first, then the rest.

Slide 1: Title (greeting)

Good morning, respected Principal, Head of the Department, our guide Prof. Chandrakala V, faculty members and friends. We are Mithila, Sneha and Yashaswini, final-year students of Information Science and Engineering. Today we present our Major Project Phase II: **AI-Powered Resume Screening and Ranking System**.

Slide 2: Agenda

We will go through the introduction, literature survey, problem statement and objectives, then methodology, architecture, implementation, data flow and interface, then the test cases and results, and finish with the conclusion and references.

Slide 3: Introduction

Hiring teams receive hundreds of resumes for every opening, and screening them by hand is slow and inconsistent. Most applicant tracking systems only count keywords, so they reward keyword stuffing and cannot see that FastAPI and Flask are related skills. Automated screening is also under scrutiny for bias, and the EU AI Act treats recruitment AI as high-risk, so every decision must be explainable. We built a system that ranks candidates on seven weighted signals, hides identity while scoring, and gives a reason for every verdict.

Slide 4: Problem statement

Keyword tools only tell whether the words of the job description appear in a resume. They cannot judge whether a skill claim is backed by evidence, they give no reason when a candidate is rejected, and they can carry hidden bias from names and personal details. So recruiters lose good candidates and cannot justify their decisions. We need a system that is accurate, explainable and bias-aware.

Slides 5-8: Literature survey

We studied ten papers, listed here from oldest to newest. They fall into four groups.

- Matching resumes to jobs: Qin et al. (2018) match abilities to jobs with a neural network; Bhatia et al. (2019) use BERT to parse resumes; Reimers and Gurevych (2019) give fast sentence embeddings; Deshmukh and Dahake (2025) compare BERT and Sentence-BERT.
- Bias in hiring: Raghavan et al. (2020), Armstrong et al. (2024) and Wilson and Caliskan (2024) show that hiring algorithms and language models can be biased; Zehlike et al. (2017) give a fair ranking method.
- Survey: Qin et al. (2023) survey AI for talent analytics.
- Multi-agent review: Du et al. (2024) show that agents debating each other improve reasoning.

The common drawback, in the last column, is that they give a score without a reason, or they audit bias without building a system. Our project combines matching, explanation and bias control in one working system.

Slide 9: Objectives

Five objectives. Design an explainable workflow that shows why each candidate was ranked and which skills are missing. Implement a hybrid ranker over seven signals. Build a skill ontology with skill-gap analysis. Minimise bias with a blind-screening mode. And give fast, auditable decisions through a four-agent review council and a FastAPI service.

Slide 10: Methodology

Eight stages. The recruiter gives a job description and resumes in PDF, DOCX, TXT or scanned form. They are parsed into skills, experience, education and projects. Blind screening hides names and personal details. The skill ontology matches synonyms and partial skills. The hybrid ranker gives a match percentage from seven weighted scores. The council of four agents reviews and debates. We get an explainable verdict with red flags, shown on the recruiter dashboard. Every stage is logged, so any ranking can be audited later.

Slide 11: Architectural design

Four layers. The presentation layer is the React web app on Vercel, the Streamlit workspace and the recruiter dashboard. The service layer is a FastAPI service on Render with secure sign-in: hashed passwords, seven-day sessions, and Google sign-in verified on the server. The intelligence engine holds the parser, ontology, hybrid ranker, council and interview kit. The data layer is MongoDB Atlas for accounts, sessions and candidate pools, plus SQLite for screening history and job roles.

Slide 12: Implementation

How a screening runs. The job description and resumes are parsed into structured profiles. Blind screening, if on, replaces names with Candidate numbers. The ontology feeds a ranker that blends

seven signals. The four agents give a verdict. The output is the leaderboard, the skill gap, a 20-question interview kit and a chatbot.

Slide 13: Hybrid ranking model

Match percentage is the weighted sum of seven scores, each from 0 to 100. Skills carry 35 percent, and related skills get partial credit, for example FastAPI through Flask counts half. Semantic fit carries 20 percent, using TF-IDF and BM25 offline, and Sentence-BERT when installed. A skill that is listed but never shown in a project counts for less. Experience deficit, skill stuffing and date inconsistency lower the verdict.

Slide 14: Four-agent council

Four agents judge each candidate. The Tech Lead checks skills and evidence, Culture judges stability, the Fair Auditor checks the decision is neutral, and Seniority compares experience with the role level. They cross-examine each other and give one verdict, Strong Hire, Lean Hire or Reject, with red flags and a debate transcript. The chart shows the agent scores and the consensus for our gold-standard candidate.

Slide 15: Data flow diagram, resume screening

Read the boxes top to bottom. The recruiter signs in and uploads the job description and resumes. They are parsed, optionally made blind, scored on seven signals, reviewed by the council, and shown on the dashboard, while the API saves the run in the screening history.

Slide 16: Data flow diagram, council decision

Inside the council. The scored profile, anonymised if blind mode is on, goes to four agents who score it independently. They debate, a consensus score and confidence are calculated, and the verdict is issued with red flags and a rationale. If confidence is low, the case is marked for human review, so the system supports the recruiter and does not replace them.

Slide 17: Test cases with results

We ran 8 test cases on the real pipeline, with no mock data, and all 8 passed. The gold-standard resume ranked first at 88.1 percent and Strong Hire. The buzzword resume, with 35 listed skills and 4 months of experience, was rejected and flagged. The lateral match got partial credit for FastAPI through Flask. The corrupt-format resume got a low ATS score and a date flag. The next four cover a second strong candidate, a junior candidate, a resume with missing sections, and a fairness test where changing the name changed no score. Cases 1 to 4 are also in our automated suite of 78 passing tests.

Slide 18: Test cases, continued

Phase 1 against Phase 2. The old keyword system gave the gold-standard resume a perfect 100, which is inflated, and gave the corrupt resume 0. Our hybrid score is 88.1 and 34.0, and the council adds a further review. A name swap changed no score, and removing evidence weighting would have turned two Reject verdicts into Lean Hire, so the evidence check matters.

Slide 19: Interface design

This is the live system. On the left is the sign-in page of the deployed website; on the right is job management, where the recruiter enters the role and its required and preferred skills.

Slide 20: Results

The leaderboard and blind mode. Each rank shows verified evidence for the skills, and with blind screening on, identities are hidden so the recruiter judges only on merit.

Slide 21: Recruiter copilot

A chatbot that answers only from the screened pool. On the left is a red-flag audit of the pool; on the right a follow-up question that is understood in context, with clickable suggestions.

Slide 22: Speed and robustness

About 30 milliseconds per resume, fully offline. Over 200 random changes to the scoring weights, the top candidate stayed first every time, with an average rank correlation of 0.97. Swapping names changed no score. The rank correlation with the old keyword baseline is only 0.67, so the ranking is really different.

Slide 23: Benchmark pool of 50 resumes

We screened 50 synthetic resumes for a Senior Machine Learning Engineer role. The top match is 93.2 percent, with zero red flags. The average is 56.9 percent. The verdicts were 10 Strong Hire, 10 Lean Hire and 30 Reject, and 30 resumes carried at least one red flag. This data is synthetic, so it shows consistency, not real-world hiring accuracy.

Slide 24: Conclusion

Resume screening can be fast, explainable and fair at the same time. The system ranks on seven signals, understands related skills, hides identity before scoring, and gives a reason for every verdict. It is deployed as a working web service.

Slide 25: Future scope

We can add role-based access, candidate consent and data-retention controls; validate the weights on real labelled hiring data; extend the evidence checks to LinkedIn, portfolios and more languages; and add email and ATS integrations with a load test for production use.

Slide 26: References

All ten papers from the literature survey, in the same order.

Slide 27: Thank you

Thank you, madam, and thank you all. We are happy to answer your questions.

Likely questions

- **Why not just use an LLM?** The core runs offline and deterministic; the LLM is optional, and every verdict must be explainable.
- **Is the data real?** No. All 162 resumes are synthetic, so results show consistency, not real hiring accuracy.
- **How is bias reduced?** Blind screening, a Fair Auditor agent, and the name-swap test showing zero score change.
- **How is Google sign-in secured?** The server verifies the Google token (audience, issuer, expiry, verified Gmail) before creating a session.